

Ereditarietà in C++

Ereditarietà in C++

Introduzione all'ereditarietà in C++, come creare classi derivate da classi esistenti.

Cos'è l'ereditarietà?

Immagina di dover creare le classi `Cane`, `Gatto` e `Coniglio`. Tutte e tre hanno un nome, un'età, mangiano e dormono. Riscrivere le stesse cose tre volte è una perdita di tempo.

L'**ereditarietà** risolve questo: scrivi una classe `Animale` con le caratteristiche comuni, e poi `Cane`, `Gatto` e `Coniglio` **ereditano** tutto da essa, aggiungendo solo quello che hanno in più.

Come in natura: un cane è un animale. Ha tutto quello che ha un animale, più le caratteristiche proprie del cane.

Come funziona l'ereditarietà

La classe "figlia" (derivata) eredita tutto dalla classe "madre" (base):

```

// Classe base (la "madre")
class Animale {
public:
    string nome;
    int eta;

    void mangia() {
        cout << nome << " sta mangiando." << endl;
    }

    void dormi() {
        cout << nome << " sta dormendo." << endl;
    }
};

// Classe derivata (la "figlia") – eredita da Animale
class Cane : public Animale {
public:
    string razza;    // campo aggiuntivo specifico del Cane

    void abbaia() {
        cout << nome << " dice: Bau!" << endl;    // usa il campo di
Animale!
    }
};

// Un'altra classe derivata
class Gatto : public Animale {
public:
    bool viveInCasa;

    void faSeLeFusa() {
        cout << nome << " fa le fusa." << endl;
    }
};

```

Usare le classi derivate

```
int main() {  
    Cane fido;  
    fido.nome = "Fido";        // ereditato da Animale  
    fido.eta = 3;  
    fido.razza = "Labrador";   // specifico di Cane  
  
    fido.mangia();             // metodo ereditato da Animale  
    fido.abbaia();             // metodo proprio di Cane  
  
    Gatto luna;  
    luna.nome = "Luna";  
    luna.dormi();              // metodo ereditato  
    luna.faSeLeFusa();         // metodo proprio di Gatto  
  
    return 0;  
}
```

Il costruttore nelle classi derivate

Quando crei un oggetto `Cane`, il C++ chiama prima il costruttore di `Animale` e poi quello di `Cane`. Per passare dati al costruttore della classe madre, usa `:` nella lista di inizializzazione:

```

class Animale {
public:
    string nome;
    int eta;

    Animale(string n, int e) : nome(n), eta(e) {
        cout << "Animale creato: " << nome << endl;
    }
};

class Cane : public Animale {
public:
    string razza;

    // Chiama il costruttore di Animale passandogli n e e
    Cane(string n, int e, string r) : Animale(n, e), razza(r) {
        cout << "Cane creato: " << nome << " (" << razza << ")" << endl;
    }
};

int main() {
    Cane fido("Fido", 3, "Labrador");
    // Stampa:
    // Animale creato: Fido
    // Cane creato: Fido (Labrador)
    return 0;
}

```

Sovrascrivere un metodo della classe madre

Una classe figlia può **ridefinire** un metodo della madre per dargli un comportamento diverso:

```

class Animale {
public:
    string nome;

    void emettiSuono() {
        cout << nome << " emette un suono generico." << endl;
    }
};

class Cane : public Animale {
public:
    // Sovrascrive il metodo della classe madre
    void emettiSuono() {
        cout << nome << " dice: Bau!" << endl;
    }
};

class Gatto : public Animale {
public:
    void emettiSuono() {
        cout << nome << " dice: Miao!" << endl;
    }
};

int main() {
    Animale a; a.nome = "Generico";
    Cane c;    c.nome = "Fido";
    Gatto g;   g.nome = "Luna";

    a.emettiSuono(); // emette un suono generico
    c.emettiSuono(); // Bau!
    g.emettiSuono(); // Miao!
    return 0;
}

```

Chiamare il metodo della classe madre

Se hai bisogno di usare anche il metodo originale della madre dentro quello della figlia:

```

class Animale {
public:
    virtual void descrivi() {
        cout << "Sono un animale." << endl;
    }
};

class Cane : public Animale {
public:
    void descrivi() {
        Animale::descrivi();    // chiama prima il metodo della madre
        cout << "Sono un cane." << endl;    // poi aggiunge qualcosa
    }
};

```

protected : il livello intermedio

private significa che nemmeno le classi figlie possono accedere al dato. A volte vuoi qualcosa di più accessibile, ma non completamente pubblico: usa **protected** :

```

class Animale {
protected:
    string nome;    // le classi figlie possono usarlo
private:
    int codiceInterno;    // le classi figlie NON possono vederlo
public:
    Animale(string n) : nome(n) {}
};

class Cane : public Animale {
public:
    Cane(string n) : Animale(n) {}

    void abbaia() {
        cout << nome << " dice: Bau!";    // OK: nome è protected
        cout << codiceInterno;    // ERRORE: è private
    }
};

```

Catene di ereditarietà

Puoi creare gerarchie con più livelli:

```
class Veicolo {
public:
    int velocitaMassima;
    void muoviti() { cout << "Mi muovo." << endl; }
};

class Auto : public Veicolo {
public:
    int numeroPosti;
};

class SportCar : public Auto {
public:
    bool haIlTurbo;
    void sgomma() { cout << "Sgommata!" << endl; }
};
```

`SportCar` eredita da `Auto`, che eredita da `Veicolo`. Quindi `SportCar` ha `velocitaMassima`, `numeroPosti`, `haIlTurbo`, più i metodi `muoviti()` e `sgomma()`.